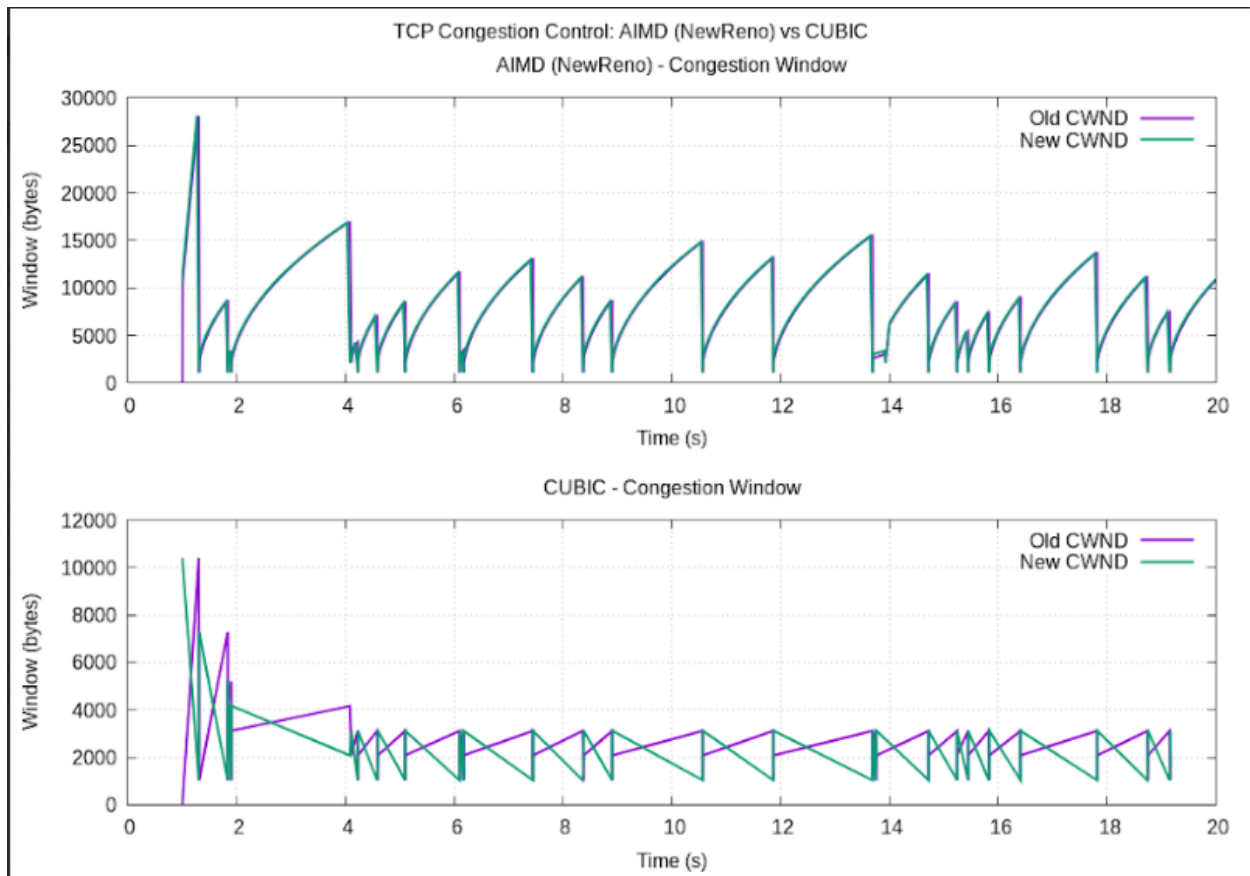
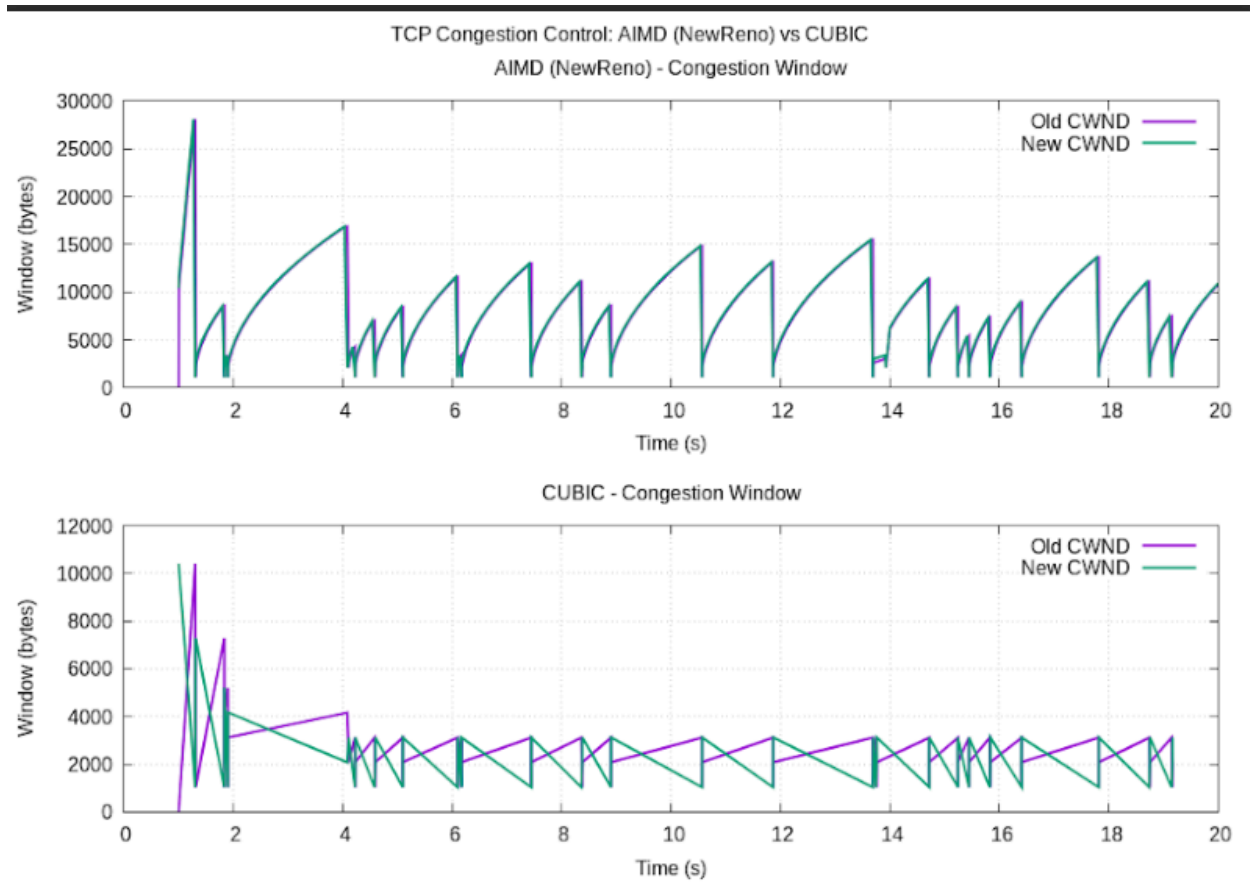


lab 8

aimd.cwnd + cubic.cwnd

```
student@student-OptiPlex-7000: $ gedit ~/tcp_congestion_analysis.py
student@student-OptiPlex-7000: $ python3 tcp_congestion_analysis.py
File "/home/student/tcp_congestion_analysis.py", line 254
    print("23K0069 John Raza")
    ^
SyntaxError: unterminated string literal (detected at line 254)
student@student-OptiPlex-7000:~$ gedit ~/tcp_congestion_analysis.py
student@student-OptiPlex-7000:~$ python3 tcp_congestion_analysis.py
=====
TCP Congestion Control: AIMD vs CUBIC Comparison
23K0069 John Raza
=====
\n[1/2] Running AIMD (TcpNewReno) simulation...
Building NS3...
\nRunning simulation with TcpNewReno...
[0/2] Re-checking globbed directories...
ninja: no work to do.
=====
TCP Variant: TcpNewReno
Output File: aimd.cwnd
Simulation Time: 20 seconds
Error Rate: 1e-05
=====
Simulation completed. Data saved to: aimd.cwnd
\n[2/2] Running CUBIC simulation...
Building NS3...
\nRunning simulation with TcpCubic...
```





```
#include <fstream>
#include <iostream>
#include <string>
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("TcpCompareExample");
```

```

class MyApp : public Application
{
public:
    MyApp ();
    virtual ~MyApp ();
    static TypeId GetTypeId (void);
    void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, DataRate dataRate);

private:
    virtual void StartApplication (void);
    virtual void StopApplication (void);
    void ScheduleTx (void);
    void SendPacket (void);

    Ptr<Socket>      m_socket;
    Address          m_peer;
    uint32_t         m_packetSize;
    uint32_t         m_nPackets;
    DataRate         m_dataRate;
    EventId          m_sendEvent;
    bool             m_running;
    uint32_t         m_packetsSent;
};

MyApp::MyApp ()
: m_socket (0),
  m_peer (),
  m_packetSize (0),
  m_nPackets (0),
  m_dataRate (0),
  m_sendEvent (),
  m_running (false),
  m_packetsSent (0)

```

```

{
}

MyApp::~MyApp ()
{
    m_socket = 0;
}

TypeId
MyApp::GetTypeId (void)
{
    static TypeId tid = TypeId ("MyApp")
        .SetParent<Application> ()
        .SetGroupName ("Tutorial")
        .AddConstructor<MyApp> ()
        ;
    return tid;
}

void
MyApp::Setup (Ptr<Socket> socket, Address address, uint32_t p
packetSize, uint32_t nPackets, DataRate dataRate)
{
    m_socket = socket;
    m_peer = address;
    m_packetSize = packetSize;
    m_nPackets = nPackets;
    m_dataRate = dataRate;
}

void
MyApp::StartApplication (void)
{
    m_running = true;
    m_packetsSent = 0;
    if (InetSocketAddress::IsMatchingType (m_peer))

```

```

    {
        m_socket->Bind ();
    }
else
    {
        m_socket->Bind6 ();
    }
m_socket->Connect (m_peer);
SendPacket ();
}

void
MyApp::StopApplication (void)
{
    m_running = false;
    if (m_sendEvent.IsRunning ())
    {
        Simulator::Cancel (m_sendEvent);
    }
    if (m_socket)
    {
        m_socket->Close ();
    }
}

void
MyApp::SendPacket (void)
{
    Ptr<Packet> packet = Create<Packet> (m_packetSize);
    m_socket->Send (packet);
    if (++m_packetsSent < m_nPackets)
    {
        ScheduleTx ();
    }
}

```

```

void
MyApp::ScheduleTx (void)
{
    if (m_running)
    {
        Time tNext (Seconds (m_packetSize * 8 / static_cast<double> (m_dataRate.GetBitRate ()))));
        m_sendEvent = Simulator::Schedule (tNext, &MyApp::SendPacket, this);
    }
}

static void
CwndChange (Ptr<OutputStreamWrapper> stream, uint32_t oldCwnd, uint32_t newCwnd)
{
    *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << oldCwnd << "\t" << newCwnd << std::endl;
}

int
main (int argc, char *argv[])
{
    std::string tcpVariant = "TcpNewReno";
    std::string outputFile = "aimd.cwnd";
    double simTime = 20.0;
    double errorRate = 0.00001;

    CommandLine cmd;
    cmd.AddValue ("tcpVariant", "TCP variant to use (TcpNewReno, TcpCubic, etc.)", tcpVariant);
    cmd.AddValue ("outputFile", "Output file name for cwnd data", outputFile);
}

```

```

    cmd.AddValue ("simTime", "Simulation duration in seconds",
simTime);
    cmd.AddValue ("errorRate", "Packet error rate for congestio
n", errorRate);
    cmd.Parse (argc, argv);

    std::cout << "=====" <<
std::endl;
    std::cout << "TCP Variant: " << tcpVariant << std::endl;
    std::cout << "Output File: " << outputFile << std::endl;
    std::cout << "Simulation Time: " << simTime << " seconds" <
< std::endl;
    std::cout << "Error Rate: " << errorRate << std::endl;
    std::cout << "=====" <<
std::endl;

    Config::SetDefault ("ns3::TcpL4Protocol::SocketType", Strin
gValue ("ns3::" + tcpVariant));

    Config::SetDefault ("ns3::TcpSocket::SegmentSize", UInteger
Value (1040));

    NodeContainer nodes;
    nodes.Create (2);

    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue
("5Mbps"));
    pointToPoint.SetChannelAttribute ("Delay", StringValue ("2m
s"));

    NetDeviceContainer devices;

```



```

devices = pointToPoint.Install (nodes);

Ptr<RateErrorModel> em = CreateObject<RateErrorModel> ();
em->SetAttribute ("ErrorRate", DoubleValue (errorRate));
devices.Get (1)->SetAttribute ("ReceiveErrorModel", Pointer
Value (em));

InternetStackHelper stack;
stack.Install (nodes);

Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (device
s);

uint16_t sinkPort = 8080;
Address sinkAddress (InetSocketAddress (interfaces.GetAddre
ss (1), sinkPort));
PacketSinkHelper packetSinkHelper ("ns3::TcpSocketFactory",
    InetSocketAddress (Ipv4Address::GetAny (), sinkPort));
ApplicationContainer sinkApps = packetSinkHelper.Install (n
odes.Get (1));
sinkApps.Start (Seconds (0.));
sinkApps.Stop (Seconds (simTime));

Ptr<Socket> ns3TcpSocket = Socket::CreateSocket (nodes.Get
(0),
    TcpSocketFactory::GetTypeId ());

Ptr<MyApp> app = CreateObject<MyApp> ();
app->Setup (ns3TcpSocket, sinkAddress, 1040, 100000, DataRa

```

```

te ("1Mbps"));
nodes.Get (0)->AddApplication (app);
app->SetStartTime (Seconds (1.));
app->SetStopTime (Seconds (simTime));

AsciiTraceHelper asciiTraceHelper;
Ptr<OutputStreamWrapper> stream = asciiTraceHelper.CreateFileStream (outputFile);
ns3TcpSocket->TraceConnectWithoutContext ("CongestionWindow",
    MakeBoundCallback (&CwndChange, stream));

Simulator::Stop (Seconds (simTime));
Simulator::Run ();
Simulator::Destroy ();

std::cout << "Simulation completed. Data saved to: " << outputFile << std::endl;

return 0;
}

```

```

#!/usr/bin/env python3
"""
TCP Congestion Control Analysis: AIMD vs CUBIC
23K-0069

"""

import subprocess

```

```

import os
import sys
import matplotlib.pyplot as plt
import numpy as np

def run_simulation(tcp_variant, output_file, sim_time=20):
    """
    Run NS3 simulation with specified TCP variant
    """
    ns3_dir = os.path.expanduser("~/ns-allinone-3.45/ns-3.45")

    print(f"Building NS3...")
    build_result = subprocess.run(
        ["/ns3", "build"],
        cwd=ns3_dir,
        capture_output=True,
        text=True
    )

    if build_result.returncode != 0:
        print(f"Build error: {build_result.stderr[:500]}")
        return False

    print(f"\nRunning simulation with {tcp_variant}...")
    cmd = [
        "/ns3", "run",
        f"scratch/tcp-compare --tcpVariant={tcp_variant} --outputFile={output_file} --simTime={sim_time}"
    ]

    result = subprocess.run(
        cmd,
        cwd=ns3_dir,

```

```

        capture_output=True,
        text=True,
        timeout=120
    )

    print(result.stdout)
    if result.stderr:
        print(f"Errors: {result.stderr[:500]}")

    return result.returncode == 0

def parse_cwnd_file(filename):
    """
    Parse cwnd data file: Time      OldCwnd      NewCwnd
    Returns: times, old_cwnds, new_cwnds
    """
    times, old_cwnds, new_cwnds = [], [], []

    if not os.path.exists(filename):
        print(f"Warning: {filename} not found!")
        return times, old_cwnds, new_cwnds

    with open(filename, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) >= 3:
                try:
                    times.append(float(parts[0]))
                    old_cwnds.append(float(parts[1]))
                    new_cwnds.append(float(parts[2]))
                except ValueError:
                    continue

    return times, old_cwnds, new_cwnds

def calculate_throughput(times, cwnds, rtt=0.004): # 4ms RTT

```

```

approx for this topology
"""
Approximate throughput from cwnd: Throughput = cwnd / RTT
"""
return [c / rtt / 1000 for c in cwnds] # kbps

def plot_comparison():
    """
    Generate comprehensive comparison plots
    """

    aimd_time, aimd_old, aimd_new = parse_cwnd_file("aimd.cwnd")
    cubic_time, cubic_old, cubic_new = parse_cwnd_file("cubic.cwnd")

    if not aimd_time or not cubic_time:
        print("Error: Could not load data files!")
        return False

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('TCP Congestion Control Comparison: AIMD (NewReno) vs CUBIC\\n23K-0842 LAB#07',
                 fontsize=14, fontweight='bold')

    ax1 = axes[0, 0]
    ax1.plot(aimd_time, aimd_new, 'b-', label='AIMD (NewReno)', linewidth=2, alpha=0.8)
    ax1.plot(cubic_time, cubic_new, 'r-', label='CUBIC', linewidth=2, alpha=0.8)
    ax1.set_xlabel('Time (seconds)')
    ax1.set_ylabel('Congestion Window (bytes)')
    ax1.set_title('Congestion Window Evolution')
    ax1.legend()

```

```

ax1.grid(True, alpha=0.3)

ax2 = axes[0, 1]
aimd_ss = [(t, c) for t, c in zip(aimd_time, aimd_new) if
t <= 2.0]
cubic_ss = [(t, c) for t, c in zip(cubic_time, cubic_new)
if t <= 2.0]

if aimd_ss and cubic_ss:
    ax2.plot([x[0] for x in aimd_ss], [x[1] for x in aimd
_ss], 'b-', label='AIMD', linewidth=2)
    ax2.plot([x[0] for x in cubic_ss], [x[1] for x in cub
ic_ss], 'r-', label='CUBIC', linewidth=2)
    ax2.set_xlabel('Time (seconds)')
    ax2.set_ylabel('Congestion Window (bytes)')
    ax2.set_title('Slow Start Phase (First 2 seconds)')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

ax3 = axes[1, 0]
aimd_tput = calculate_throughput(aimd_time, aimd_new)
cubic_tput = calculate_throughput(cubic_time, cubic_new)

ax3.plot(aimd_time, aimd_tput, 'b-', label='AIMD (NewRen
o)', linewidth=2, alpha=0.8)
ax3.plot(cubic_time, cubic_tput, 'r-', label='CUBIC', lin
ewidth=2, alpha=0.8)
ax3.set_xlabel('Time (seconds)')
ax3.set_ylabel('Throughput (kbps)')
ax3.set_title('Approximate Throughput (cwnd/RTT)')
ax3.legend()
ax3.grid(True, alpha=0.3)

```

```

    ax4 = axes[1, 1]
    ax4.hist(aimd_new, bins=30, alpha=0.5, label='AIMD', color='blue', density=True)
    ax4.hist(cubic_new, bins=30, alpha=0.5, label='CUBIC', color='red', density=True)
    ax4.set_xlabel('Congestion Window (bytes)')
    ax4.set_ylabel('Density')
    ax4.set_title('CWND Distribution')
    ax4.legend()
    ax4.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('tcp_aimd_vs_cubic_comparison.png', dpi=150,
bbox_inches='tight')
    print("\n\nPlot saved as: tcp_aimd_vs_cubic_comparison.png")
    plt.show()

    return True

def generate_gnuplot_script():
    """
    Generate gnuplot script for traditional plotting (matches
    lab format)
    """
    script = '''# Gnuplot script for TCP Congestion Control Comparison
# 23K-0069

set terminal pngcairo enhanced font 'Arial,12' size 1000,700
set output 'congestion_comparison.png'

set multiplot layout 2,1 title "TCP Congestion Control: AIMD
(NewReno) vs CUBIC"

# Plot 1: AIMD Congestion Window

```

```

set title "AIMD (NewReno) - Congestion Window"
set xlabel "Time (s)"
set ylabel "Window (bytes)"
set grid
plot "aimd.cwnd" using 1:2 with lines lw 2 title "Old CWND",
\\
    "aimd.cwnd" using 1:3 with lines lw 2 title "New CWND"

# Plot 2: CUBIC Congestion Window
set title "CUBIC - Congestion Window"
set xlabel "Time (s)"
set ylabel "Window (bytes)"
set grid
plot "cubic.cwnd" using 1:2 with lines lw 2 title "Old CWND",
\\
    "cubic.cwnd" using 1:3 with lines lw 2 title "New CWND"

unset multiplot

# Alternative: Combined comparison
set terminal pngcairo enhanced font 'Arial,12' size 1000,600
set output "combined_comparison.png"
set title "AIMD vs CUBIC Congestion Window Comparison"
set xlabel "Time (s)"
set ylabel "Congestion Window (bytes)"
set grid
plot "aimd.cwnd" using 1:3 with lines lw 2 title "AIMD (NewRe
no)", \\
    "cubic.cwnd" using 1:3 with lines lw 2 title "CUBIC"
...

with open("plot_tcp_comparison.gp", "w") as f:
    f.write(script)

print("Gnuplot script saved as: plot_tcp_comparison.gp")
print("Run with: gnuplot plot_tcp_comparison.gp")

```



```

def print_analysis():
    """
    Print performance analysis
    """
    aimd_time, aimd_old, aimd_new = parse_cwnd_file("aimd.cwnd")
    cubic_time, cubic_old, cubic_new = parse_cwnd_file("cubic.cwnd")

    print("\n" + "="*60)
    print("PERFORMANCE ANALYSIS: AIMD vs CUBIC")
    print("="*60)

    if aimd_new and cubic_new:
        print(f"\nAIMD (NewReno) Statistics:")
        print(f"  Max CWND: {max(aimd_new):.0f} bytes")
        print(f"  Avg CWND: {sum(aimd_new)/len(aimd_new):.0f} bytes")
        print(f"  Final CWND: {aimd_new[-1]:.0f} bytes")
        print(f"  Samples: {len(aimd_new)}")

        print(f"\nCUBIC Statistics:")
        print(f"  Max CWND: {max(cubic_new):.0f} bytes")
        print(f"  Avg CWND: {sum(cubic_new)/len(cubic_new):.0f} bytes")
        print(f"  Final CWND: {cubic_new[-1]:.0f} bytes")
        print(f"  Samples: {len(cubic_new)}")

        avg_aimd = sum(aimd_new)/len(aimd_new)
        avg_cubic = sum(cubic_new)/len(cubic_new)
        improvement = ((avg_cubic - avg_aimd) / avg_aimd) * 100

        print(f"\nComparison:")
        print(f"  CUBIC avg improvement: {improvement:+.1f}

```

```

f}%")
    print(f"    Max CWND ratio (CUBIC/AIMD): {max(cubic_new) / max(aimd_new):.2f}x")

    print("\n" + "="*60)
    print("KEY DIFFERENCES:")
    print("="*60)
    print("1. AIMD (NewReno):")
    print("    - Linear growth in congestion avoidance (add 1 MSS per RTT)")
    print("    - Conservative after packet loss (halve cwnd)")
    print("    - Better fairness among competing flows")
    print("    - Lower throughput in high BDP networks")
    print("")
    print("2. CUBIC:")
    print("    - Cubic function growth (concave then convex)")
    print("    - Aggressive bandwidth probing")
    print("    - Better utilization of high BDP links")
    print("    - Faster recovery to previous cwnd max")
    print("    - Default in Linux, macOS, Windows")
    print("="*60)

def main():
    print("="*60)
    print("TCP Congestion Control: AIMD vs CUBIC Comparison")
    print("23K-0842 LAB#07")
    print("="*60)

    ns3_dir = os.path.expanduser("~/ns-allinone-3.45/ns-3.45")
    if not os.path.exists(ns3_dir):
        print(f"ERROR: NS3 not found at {ns3_dir}")
        print("Please install NS3 first!")

```

```

        return 1

print("\n\n[1/2] Running AIMD (TcpNewReno) simulation...")
if not run_simulation("TcpNewReno", "aimd.cwnd", 20):
    print("AIMD simulation failed!")
    return 1

print("\n\n[2/2] Running CUBIC simulation...")
if not run_simulation("TcpCubic", "cubic.cwnd", 20):
    print("CUBIC simulation failed!")
    return 1

print("\n\n[3/3] Generating comparison plots...")
try:
    plot_comparison()
except Exception as e:
    print(f"Matplotlib plotting failed: {e}")
    print("Generating gnuplot script instead...")
    generate_gnuplot_script()

generate_gnuplot_script()

print_analysis()

print("\n\n" + "="*60)
print("ASSIGNMENT COMPLETE!")
print("="*60)
print("Generated files:")
print("  - aimd.cwnd (AIMD congestion window data)")
print("  - cubic.cwnd (CUBIC congestion window data)")
print("  - tcp_aimd_vs_cubic_comparison.png (matplotlib p

```

```

lot)")
    print(" - plot_tcp_comparison.gp (gnuplot script)")
    print(" - congestion_comparison.png (gnuplot output)")
    print("="*60)

    return 0

if __name__ == "__main__":
    sys.exit(main())

```

aimd.cwnd

```

1.00419 0 10400
1.01011 10400 11440
1.02248 11440 12480
1.03912 12480 13520
1.05576 13520 14560
1.0724 14560 15600
1.08904 15600 16640
1.10568 16640 17680
1.12232 17680 18720
1.13896 18720 19760
1.1556 19760 20800
1.17224 20800 21840
1.18888 21840 22880
1.20552 22880 23920
1.22216 23920 24960
1.2388 24960 26000
1.25544 26000 27040
1.27208 27040 28080
1.30537 28080 1040
1.31121 1040 2080
1.31121 2080 2600

```

1.322	2600	3016
1.33864	3016	3374
1.35528	3374	3694
1.37192	3694	3986
1.38856	3986	4257
1.4052	4257	4511
1.42184	4511	4750
1.43848	4750	4977
1.45512	4977	5194
1.47176	5194	5402
1.4884	5402	5602
1.50504	5602	5795
1.52168	5795	5981
1.53832	5981	6161
1.55496	6161	6336
1.5716	6336	6506
1.58824	6506	6672
1.60488	6672	6834
1.62152	6834	6992
1.63816	6992	7146
1.6548	7146	7297
1.67144	7297	7445
1.68808	7445	7590
1.70472	7590	7732
1.72136	7732	7871
1.738	7871	8008
1.75464	8008	8143
1.77128	8143	8275
1.78792	8275	8405
1.80456	8405	8533
1.83785	8533	1040
1.84369	1040	2080
1.84369	2080	2600
1.85448	2600	3016
1.87112	3016	3374
1.90441	3374	1040

1.91025	1040	2080
1.91025	2080	2600
1.92104	2600	3016
1.93768	3016	3374
1.95432	3374	3694
1.97096	3694	3986
1.9876	3986	4257
2.00424	4257	4511
2.02088	4511	4750
2.03752	4750	4977
2.05416	4977	5194
2.0708	5194	5402
2.08744	5402	5602
2.10408	5602	5795
2.12072	5795	5981
2.13736	5981	6161
2.154	6161	6336
2.17064	6336	6506
2.18728	6506	6672
2.20392	6672	6834
2.22056	6834	6992
2.2372	6992	7146
2.25384	7146	7297
2.27048	7297	7445
2.28712	7445	7590
2.30376	7590	7732
2.3204	7732	7871
2.33704	7871	8008
2.35368	8008	8143
2.37032	8143	8275
2.38696	8275	8405
2.4036	8405	8533
2.42024	8533	8659
2.43688	8659	8783
2.45352	8783	8906
2.47016	8906	9027

2.4868	9027	9146
2.50344	9146	9264
2.52008	9264	9380
2.53672	9380	9495
2.55336	9495	9608
2.57	9608	9720
2.58664	9720	9831
2.60328	9831	9941
2.61992	9941	10049
2.63656	10049	10156
2.6532	10156	10262
2.66984	10262	10367
2.68648	10367	10471
2.70312	10471	10574
2.71976	10574	10676
2.7364	10676	10777
2.75304	10777	10877
2.76968	10877	10976
2.78632	10976	11074
2.80296	11074	11171
2.8196	11171	11267
2.83624	11267	11362
2.85288	11362	11457
2.86952	11457	11551
2.88616	11551	11644
2.9028	11644	11736
2.91944	11736	11828
2.93608	11828	11919
2.95272	11919	12009
2.96936	12009	12099
2.986	12099	12188
3.00264	12188	12276
3.01928	12276	12364
3.03592	12364	12451
3.05256	12451	12537
3.0692	12537	12623

3.08584	12623	12708
3.10248	12708	12793
3.11912	12793	12877
3.13576	12877	12960
3.1524	12960	13043
3.16904	13043	13125
3.18568	13125	13207
3.20232	13207	13288
3.21896	13288	13369
3.2356	13369	13449
3.25224	13449	13529
3.26888	13529	13608
3.28552	13608	13687
3.30216	13687	13766
3.3188	13766	13844
3.33544	13844	13922
3.35208	13922	13999
3.36872	13999	14076
3.38536	14076	14152
3.402	14152	14228
3.41864	14228	14304
3.43528	14304	14379
3.45192	14379	14454
3.46856	14454	14528
3.4852	14528	14602
3.50184	14602	14676
3.51848	14676	14749
3.53512	14749	14822
3.55176	14822	14894
3.5684	14894	14966
3.58504	14966	15038
3.60168	15038	15109
3.61832	15109	15180
3.63496	15180	15251
3.6516	15251	15321
3.66824	15321	15391

3.68488	15391	15461
3.70152	15461	15530
3.71816	15530	15599
3.7348	15599	15668
3.75144	15668	15737
3.76808	15737	15805
3.78472	15805	15873
3.80136	15873	15941
3.818	15941	16008
3.83464	16008	16075
3.85128	16075	16142
3.86792	16142	16209
3.88456	16209	16275
3.9012	16275	16341
3.91784	16341	16407
3.93448	16407	16472
3.95112	16472	16537
3.96776	16537	16602
3.9844	16602	16667
4.00104	16667	16731
4.01768	16731	16795
4.03432	16795	16859
4.08426	16859	2080
4.10765	2080	2600
4.11752	2600	3016
4.13416	3016	3374
4.1508	3374	3694
4.16744	3694	3986
4.18408	3986	4257
4.22569	4257	1040
4.23153	1040	2080
4.23153	2080	2600
4.24232	2600	3016
4.25896	3016	3374
4.2756	3374	3694
4.29224	3694	3986

4.30888	3986	4257
4.32552	4257	4511
4.34216	4511	4750
4.3588	4750	4977
4.37544	4977	5194
4.39208	5194	5402
4.40872	5402	5602
4.42536	5602	5795
4.442	5795	5981
4.45864	5981	6161
4.47528	6161	6336
4.49192	6336	6506
4.50856	6506	6672
4.5252	6672	6834
4.54184	6834	6992
4.58345	6992	1040
4.58929	1040	2080
4.58929	2080	2600
4.60008	2600	3016
4.61672	3016	3374
4.63336	3374	3694
4.65	3694	3986
4.66664	3986	4257
4.68328	4257	4511
4.69992	4511	4750
4.71656	4750	4977
4.7332	4977	5194
4.74984	5194	5402
4.76648	5402	5602
4.78312	5602	5795
4.79976	5795	5981
4.8164	5981	6161
4.83304	6161	6336
4.84968	6336	6506
4.86632	6506	6672
4.88296	6672	6834

4.8996	6834	6992
4.91624	6992	7146
4.93288	7146	7297
4.94952	7297	7445
4.96616	7445	7590
4.9828	7590	7732
4.99944	7732	7871
5.01608	7871	8008
5.03272	8008	8143
5.04936	8143	8275
5.066	8275	8405
5.09929	8405	1040
5.10513	1040	2080
5.10513	2080	2600
5.11592	2600	3016
5.13256	3016	3374
5.1492	3374	3694
5.16584	3694	3986
5.18248	3986	4257
5.19912	4257	4511
5.21576	4511	4750
5.2324	4750	4977
5.24904	4977	5194
5.26568	5194	5402
5.28232	5402	5602
5.29896	5602	5795
5.3156	5795	5981
5.33224	5981	6161
5.34888	6161	6336
5.36552	6336	6506
5.38216	6506	6672
5.3988	6672	6834
5.41544	6834	6992
5.43208	6992	7146
5.44872	7146	7297
5.46536	7297	7445

5.482	7445	7590
5.49864	7590	7732
5.51528	7732	7871
5.53192	7871	8008
5.54856	8008	8143
5.5652	8143	8275
5.58184	8275	8405
5.59848	8405	8533
5.61512	8533	8659
5.63176	8659	8783
5.6484	8783	8906
5.66504	8906	9027
5.68168	9027	9146
5.69832	9146	9264
5.71496	9264	9380
5.7316	9380	9495
5.74824	9495	9608
5.76488	9608	9720
5.78152	9720	9831
5.79816	9831	9941
5.8148	9941	10049
5.83144	10049	10156
5.84808	10156	10262
5.86472	10262	10367
5.88136	10367	10471
5.898	10471	10574
5.91464	10574	10676
5.93128	10676	10777
5.94792	10777	10877
5.96456	10877	10976
5.9812	10976	11074
5.99784	11074	11171
6.01448	11171	11267
6.03112	11267	11362
6.04776	11362	11457
6.0644	11457	11551

6.09769	11551	1040
6.10353	1040	2080
6.10353	2080	2600
6.11432	2600	3016
6.13096	3016	3374
6.17257	3374	1040
6.17841	1040	2080
6.17841	2080	2600
6.1892	2600	3016
6.20584	3016	3374
6.22248	3374	3694
6.23912	3694	3986
6.25576	3986	4257
6.2724	4257	4511
6.28904	4511	4750
6.30568	4750	4977
6.32232	4977	5194
6.33896	5194	5402
6.3556	5402	5602
6.37224	5602	5795
6.38888	5795	5981
6.40552	5981	6161
6.42216	6161	6336
6.4388	6336	6506
6.45544	6506	6672
6.47208	6672	6834
6.48872	6834	6992
6.50536	6992	7146
6.522	7146	7297
6.53864	7297	7445
6.55528	7445	7590
6.57192	7590	7732
6.58856	7732	7871
6.6052	7871	8008
6.62184	8008	8143
6.63848	8143	8275

6.65512	8275	8405
6.67176	8405	8533
6.6884	8533	8659
6.70504	8659	8783
6.72168	8783	8906
6.73832	8906	9027
6.75496	9027	9146
6.7716	9146	9264
6.78824	9264	9380
6.80488	9380	9495
6.82152	9495	9608
6.83816	9608	9720
6.8548	9720	9831
6.87144	9831	9941
6.88808	9941	10049
6.90472	10049	10156
6.92136	10156	10262
6.938	10262	10367
6.95464	10367	10471
6.97128	10471	10574
6.98792	10574	10676
7.00456	10676	10777
7.0212	10777	10877
7.03784	10877	10976
7.05448	10976	11074
7.07112	11074	11171
7.08776	11171	11267
7.1044	11267	11362
7.12104	11362	11457
7.13768	11457	11551
7.15432	11551	11644
7.17096	11644	11736
7.1876	11736	11828
7.20424	11828	11919
7.22088	11919	12009
7.23752	12009	12099

7.25416	12099	12188
7.2708	12188	12276
7.28744	12276	12364
7.30408	12364	12451
7.32072	12451	12537
7.33736	12537	12623
7.354	12623	12708
7.37064	12708	12793
7.38728	12793	12877
7.40392	12877	12960
7.44553	12960	1040
7.45137	1040	2080
7.45137	2080	2600
7.46216	2600	3016
7.4788	3016	3374
7.49544	3374	3694
7.51208	3694	3986
7.52872	3986	4257
7.54536	4257	4511
7.562	4511	4750
7.57864	4750	4977
7.59528	4977	5194
7.61192	5194	5402
7.62856	5402	5602
7.6452	5602	5795
7.66184	5795	5981
7.67848	5981	6161
7.69512	6161	6336
7.71176	6336	6506
7.7284	6506	6672
7.74504	6672	6834
7.76168	6834	6992
7.77832	6992	7146
7.79496	7146	7297
7.8116	7297	7445
7.82824	7445	7590

7.84488	7590	7732
7.86152	7732	7871
7.87816	7871	8008
7.8948	8008	8143
7.91144	8143	8275
7.92808	8275	8405
7.94472	8405	8533
7.96136	8533	8659
7.978	8659	8783
7.99464	8783	8906
8.01128	8906	9027
8.02792	9027	9146
8.04456	9146	9264
8.0612	9264	9380
8.07784	9380	9495
8.09448	9495	9608
8.11112	9608	9720
8.12776	9720	9831
8.1444	9831	9941
8.16104	9941	10049
8.17768	10049	10156
8.19432	10156	10262
8.21096	10262	10367
8.2276	10367	10471
8.24424	10471	10574
8.26088	10574	10676
8.27752	10676	10777
8.29416	10777	10877
8.3108	10877	10976
8.32744	10976	11074
8.36073	11074	1040
8.36657	1040	2080
8.36657	2080	2600
8.37736	2600	3016
8.394	3016	3374
8.41064	3374	3694

8.42728	3694	3986
8.44392	3986	4257
8.46056	4257	4511
8.4772	4511	4750
8.49384	4750	4977
8.51048	4977	5194
8.52712	5194	5402
8.54376	5402	5602
8.5604	5602	5795
8.57704	5795	5981
8.59368	5981	6161
8.61032	6161	6336
8.62696	6336	6506
8.6436	6506	6672
8.66024	6672	6834
8.67688	6834	6992
8.69352	6992	7146
8.71016	7146	7297
8.7268	7297	7445
8.74344	7445	7590
8.76008	7590	7732
8.77672	7732	7871
8.79336	7871	8008
8.81	8008	8143
8.82664	8143	8275
8.84328	8275	8405
8.85992	8405	8533
8.90153	8533	1040
8.90737	1040	2080
8.90737	2080	2600
8.91816	2600	3016
8.9348	3016	3374
8.95144	3374	3694
8.96808	3694	3986
8.98472	3986	4257
9.00136	4257	4511

9.018	4511	4750
9.03464	4750	4977
9.05128	4977	5194
9.06792	5194	5402
9.08456	5402	5602
9.1012	5602	5795
9.11784	5795	5981
9.13448	5981	6161
9.15112	6161	6336
9.16776	6336	6506
9.1844	6506	6672
9.20104	6672	6834
9.21768	6834	6992
9.23432	6992	7146
9.25096	7146	7297
9.2676	7297	7445
9.28424	7445	7590
9.30088	7590	7732
9.31752	7732	7871
9.33416	7871	8008
9.3508	8008	8143
9.36744	8143	8275
9.38408	8275	8405
9.40072	8405	8533
9.41736	8533	8659
9.434	8659	8783
9.45064	8783	8906
9.46728	8906	9027
9.48392	9027	9146
9.50056	9146	9264
9.5172	9264	9380
9.53384	9380	9495
9.55048	9495	9608
9.56712	9608	9720
9.58376	9720	9831
9.6004	9831	9941

9.61704	9941	10049
9.63368	10049	10156
9.65032	10156	10262
9.66696	10262	10367
9.6836	10367	10471
9.70024	10471	10574
9.71688	10574	10676
9.73352	10676	10777
9.75016	10777	10877
9.7668	10877	10976
9.78344	10976	11074
9.80008	11074	11171
9.81672	11171	11267
9.83336	11267	11362
9.85	11362	11457
9.86664	11457	11551
9.88328	11551	11644
9.89992	11644	11736
9.91656	11736	11828
9.9332	11828	11919
9.94984	11919	12009
9.96648	12009	12099
9.98312	12099	12188
9.99976	12188	12276
10.0164	12276	12364
10.033	12364	12451
10.0497	12451	12537
10.0663	12537	12623
10.083	12623	12708
10.0996	12708	12793
10.1162	12793	12877
10.1329	12877	12960
10.1495	12960	13043
10.1662	13043	13125
10.1828	13125	13207
10.1994	13207	13288

10.2161	13288	13369
10.2327	13369	13449
10.2494	13449	13529
10.266	13529	13608
10.2826	13608	13687
10.2993	13687	13766
10.3159	13766	13844
10.3326	13844	13922
10.3492	13922	13999
10.3658	13999	14076
10.3825	14076	14152
10.3991	14152	14228
10.4158	14228	14304
10.4324	14304	14379
10.449	14379	14454
10.4657	14454	14528
10.4823	14528	14602
10.499	14602	14676
10.5156	14676	14749
10.5322	14749	14822
10.5655	14822	1040
10.5714	1040	2080
10.5714	2080	2600
10.5822	2600	3016
10.5988	3016	3374
10.6154	3374	3694
10.6321	3694	3986
10.6487	3986	4257
10.6654	4257	4511
10.682	4511	4750
10.6986	4750	4977
10.7153	4977	5194
10.7319	5194	5402
10.7486	5402	5602
10.7652	5602	5795
10.7818	5795	5981

10.7985	5981	6161
10.8151	6161	6336
10.8318	6336	6506
10.8484	6506	6672
10.865	6672	6834
10.8817	6834	6992
10.8983	6992	7146
10.915	7146	7297
10.9316	7297	7445
10.9482	7445	7590
10.9649	7590	7732
10.9815	7732	7871
10.9982	7871	8008
11.0148	8008	8143
11.0314	8143	8275
11.0481	8275	8405
11.0647	8405	8533
11.0814	8533	8659
11.098	8659	8783
11.1146	8783	8906
11.1313	8906	9027
11.1479	9027	9146
11.1646	9146	9264
11.1812	9264	9380
11.1978	9380	9495
11.2145	9495	9608
11.2311	9608	9720
11.2478	9720	9831
11.2644	9831	9941
11.281	9941	10049
11.2977	10049	10156
11.3143	10156	10262
11.331	10262	10367
11.3476	10367	10471
11.3642	10471	10574
11.3809	10574	10676

11.3975	10676	10777
11.4142	10777	10877
11.4308	10877	10976
11.4474	10976	11074
11.4641	11074	11171
11.4807	11171	11267
11.4974	11267	11362
11.514	11362	11457
11.5306	11457	11551
11.5473	11551	11644
11.5639	11644	11736
11.5806	11736	11828
11.5972	11828	11919
11.6138	11919	12009
11.6305	12009	12099
11.6471	12099	12188
11.6638	12188	12276
11.6804	12276	12364
11.697	12364	12451
11.7137	12451	12537
11.7303	12537	12623
11.747	12623	12708
11.7636	12708	12793
11.7802	12793	12877
11.7969	12877	12960
11.8135	12960	13043
11.8302	13043	13125
11.8634	13125	1040
11.8693	1040	2080
11.8693	2080	2600
11.8801	2600	3016
11.8967	3016	3374
11.9134	3374	3694
11.93	3694	3986
11.9466	3986	4257
11.9633	4257	4511

11.9799	4511	4750
11.9966	4750	4977
12.0132	4977	5194
12.0298	5194	5402
12.0465	5402	5602
12.0631	5602	5795
12.0798	5795	5981
12.0964	5981	6161
12.113	6161	6336
12.1297	6336	6506
12.1463	6506	6672
12.163	6672	6834
12.1796	6834	6992
12.1962	6992	7146
12.2129	7146	7297
12.2295	7297	7445
12.2462	7445	7590
12.2628	7590	7732
12.2794	7732	7871
12.2961	7871	8008
12.3127	8008	8143
12.3294	8143	8275
12.346	8275	8405
12.3626	8405	8533
12.3793	8533	8659
12.3959	8659	8783
12.4126	8783	8906
12.4292	8906	9027
12.4458	9027	9146
12.4625	9146	9264
12.4791	9264	9380
12.4958	9380	9495
12.5124	9495	9608
12.529	9608	9720
12.5457	9720	9831
12.5623	9831	9941

12.579	9941	10049
12.5956	10049	10156
12.6122	10156	10262
12.6289	10262	10367
12.6455	10367	10471
12.6622	10471	10574
12.6788	10574	10676
12.6954	10676	10777
12.7121	10777	10877
12.7287	10877	10976
12.7454	10976	11074
12.762	11074	11171
12.7786	11171	11267
12.7953	11267	11362
12.8119	11362	11457
12.8286	11457	11551
12.8452	11551	11644
12.8618	11644	11736
12.8785	11736	11828
12.8951	11828	11919
12.9118	11919	12009
12.9284	12009	12099
12.945	12099	12188
12.9617	12188	12276
12.9783	12276	12364
12.995	12364	12451
13.0116	12451	12537
13.0282	12537	12623
13.0449	12623	12708
13.0615	12708	12793
13.0782	12793	12877
13.0948	12877	12960
13.1114	12960	13043
13.1281	13043	13125
13.1447	13125	13207
13.1614	13207	13288

13.178	13288	13369
13.1946	13369	13449
13.2113	13449	13529
13.2279	13529	13608
13.2446	13608	13687
13.2612	13687	13766
13.2778	13766	13844
13.2945	13844	13922
13.3111	13922	13999
13.3278	13999	14076
13.3444	14076	14152
13.361	14152	14228
13.3777	14228	14304
13.3943	14304	14379
13.411	14379	14454
13.4276	14454	14528
13.4442	14528	14602
13.4609	14602	14676
13.4775	14676	14749
13.4942	14749	14822
13.5108	14822	14894
13.5274	14894	14966
13.5441	14966	15038
13.5607	15038	15109
13.5774	15109	15180
13.594	15180	15251
13.6106	15251	15321
13.6273	15321	15391
13.6439	15391	15461
13.6855	15461	1040
13.6914	1040	2080
13.6914	2080	2600
13.7022	2600	3016
13.9105	3016	3374
13.9222	3374	2080
13.928	2080	2600

13.9338	2600	3016
13.9414	3016	3374
13.949	3374	3694
13.9549	3694	3986
13.9607	3986	4257
13.9665	4257	4511
13.97	4511	4750
13.9741	4750	4977
13.9776	4977	5194
13.9817	5194	5402
13.9852	5402	5602
13.9893	5602	5795
13.9928	5795	5981
13.9963	5981	6161
14.0017	6161	6336
14.0183	6336	6506
14.035	6506	6672
14.0516	6672	6834
14.0682	6834	6992
14.0849	6992	7146
14.1015	7146	7297
14.1182	7297	7445
14.1348	7445	7590
14.1514	7590	7732
14.1681	7732	7871
14.1847	7871	8008
14.2014	8008	8143
14.218	8143	8275
14.2346	8275	8405
14.2513	8405	8533
14.2679	8533	8659
14.2846	8659	8783
14.3012	8783	8906
14.3178	8906	9027
14.3345	9027	9146
14.3511	9146	9264

14.3678	9264	9380
14.3844	9380	9495
14.401	9495	9608
14.4177	9608	9720
14.4343	9720	9831
14.451	9831	9941
14.4676	9941	10049
14.4842	10049	10156
14.5009	10156	10262
14.5175	10262	10367
14.5342	10367	10471
14.5508	10471	10574
14.5674	10574	10676
14.5841	10676	10777
14.6007	10777	10877
14.6174	10877	10976
14.634	10976	11074
14.6506	11074	11171
14.6673	11171	11267
14.6839	11267	11362
14.7172	11362	1040
14.723	1040	2080
14.723	2080	2600
14.7338	2600	3016
14.7505	3016	3374
14.7671	3374	3694
14.7838	3694	3986
14.8004	3986	4257
14.817	4257	4511
14.8337	4511	4750
14.8503	4750	4977
14.867	4977	5194
14.8836	5194	5402
14.9002	5402	5602
14.9169	5602	5795
14.9335	5795	5981

14.9502	5981	6161
14.9668	6161	6336
14.9834	6336	6506
15.0001	6506	6672
15.0167	6672	6834
15.0334	6834	6992
15.05	6992	7146
15.0666	7146	7297
15.0833	7297	7445
15.0999	7445	7590
15.1166	7590	7732
15.1332	7732	7871
15.1498	7871	8008
15.1665	8008	8143
15.1831	8143	8275
15.1998	8275	8405
15.2414	8405	1040
15.2472	1040	2080
15.2472	2080	2600
15.258	2600	3016
15.2746	3016	3374
15.2913	3374	3694
15.3079	3694	3986
15.3246	3986	4257
15.3412	4257	4511
15.3578	4511	4750
15.3745	4750	4977
15.3911	4977	5194
15.4078	5194	5402
15.441	5402	1040
15.4469	1040	2080
15.4469	2080	2600
15.4577	2600	3016
15.4743	3016	3374
15.491	3374	3694
15.5076	3694	3986

15.5242	3986	4257
15.5409	4257	4511
15.5575	4511	4750
15.5742	4750	4977
15.5908	4977	5194
15.6074	5194	5402
15.6241	5402	5602
15.6407	5602	5795
15.6574	5795	5981
15.674	5981	6161
15.6906	6161	6336
15.7073	6336	6506
15.7239	6506	6672
15.7406	6672	6834
15.7572	6834	6992
15.7738	6992	7146
15.7905	7146	7297
15.8238	7297	1040
15.8296	1040	2080
15.8296	2080	2600
15.8404	2600	3016
15.857	3016	3374
15.8737	3374	3694
15.8903	3694	3986
15.907	3986	4257
15.9236	4257	4511
15.9402	4511	4750
15.9569	4750	4977
15.9735	4977	5194
15.9902	5194	5402
16.0068	5402	5602
16.0234	5602	5795
16.0401	5795	5981
16.0567	5981	6161
16.0734	6161	6336
16.09	6336	6506

16.1066	6506	6672
16.1233	6672	6834
16.1399	6834	6992
16.1566	6992	7146
16.1732	7146	7297
16.1898	7297	7445
16.2065	7445	7590
16.2231	7590	7732
16.2398	7732	7871
16.2564	7871	8008
16.273	8008	8143
16.2897	8143	8275
16.3063	8275	8405
16.323	8405	8533
16.3396	8533	8659
16.3562	8659	8783
16.3729	8783	8906
16.4062	8906	1040
16.412	1040	2080
16.412	2080	2600
16.4228	2600	3016
16.4394	3016	3374
16.4561	3374	3694
16.4727	3694	3986
16.4894	3986	4257
16.506	4257	4511
16.5226	4511	4750
16.5393	4750	4977
16.5559	4977	5194
16.5726	5194	5402
16.5892	5402	5602
16.6058	5602	5795
16.6225	5795	5981
16.6391	5981	6161
16.6558	6161	6336
16.6724	6336	6506

16.689	6506	6672
16.7057	6672	6834
16.7223	6834	6992
16.739	6992	7146
16.7556	7146	7297
16.7722	7297	7445
16.7889	7445	7590
16.8055	7590	7732
16.8222	7732	7871
16.8388	7871	8008
16.8554	8008	8143
16.8721	8143	8275
16.8887	8275	8405
16.9054	8405	8533
16.922	8533	8659
16.9386	8659	8783
16.9553	8783	8906
16.9719	8906	9027
16.9886	9027	9146
17.0052	9146	9264
17.0218	9264	9380
17.0385	9380	9495
17.0551	9495	9608
17.0718	9608	9720
17.0884	9720	9831
17.105	9831	9941
17.1217	9941	10049
17.1383	10049	10156
17.155	10156	10262
17.1716	10262	10367
17.1882	10367	10471
17.2049	10471	10574
17.2215	10574	10676
17.2382	10676	10777
17.2548	10777	10877
17.2714	10877	10976

17.2881	10976	11074
17.3047	11074	11171
17.3214	11171	11267
17.338	11267	11362
17.3546	11362	11457
17.3713	11457	11551
17.3879	11551	11644
17.4046	11644	11736
17.4212	11736	11828
17.4378	11828	11919
17.4545	11919	12009
17.4711	12009	12099
17.4878	12099	12188
17.5044	12188	12276
17.521	12276	12364
17.5377	12364	12451
17.5543	12451	12537
17.571	12537	12623
17.5876	12623	12708
17.6042	12708	12793
17.6209	12793	12877
17.6375	12877	12960
17.6542	12960	13043
17.6708	13043	13125
17.6874	13125	13207
17.7041	13207	13288
17.7207	13288	13369
17.7374	13369	13449
17.754	13449	13529
17.7706	13529	13608
17.8122	13608	1040
17.8181	1040	2080
17.8181	2080	2600
17.8289	2600	3016
17.8455	3016	3374
17.8622	3374	3694

17.8788	3694	3986
17.8954	3986	4257
17.9121	4257	4511
17.9287	4511	4750
17.9454	4750	4977
17.962	4977	5194
17.9786	5194	5402
17.9953	5402	5602
18.0119	5602	5795
18.0286	5795	5981
18.0452	5981	6161
18.0618	6161	6336
18.0785	6336	6506
18.0951	6506	6672
18.1118	6672	6834
18.1284	6834	6992
18.145	6992	7146
18.1617	7146	7297
18.1783	7297	7445
18.195	7445	7590
18.2116	7590	7732
18.2282	7732	7871
18.2449	7871	8008
18.2615	8008	8143
18.2782	8143	8275
18.2948	8275	8405
18.3114	8405	8533
18.3281	8533	8659
18.3447	8659	8783
18.3614	8783	8906
18.378	8906	9027
18.3946	9027	9146
18.4113	9146	9264
18.4279	9264	9380
18.4446	9380	9495
18.4612	9495	9608

18.4778	9608	9720
18.4945	9720	9831
18.5111	9831	9941
18.5278	9941	10049
18.5444	10049	10156
18.561	10156	10262
18.5777	10262	10367
18.5943	10367	10471
18.611	10471	10574
18.6276	10574	10676
18.6442	10676	10777
18.6609	10777	10877
18.6775	10877	10976
18.6942	10976	11074
18.7358	11074	1040
18.7416	1040	2080
18.7416	2080	2600
18.7524	2600	3016
18.769	3016	3374
18.7857	3374	3694
18.8023	3694	3986
18.819	3986	4257
18.8356	4257	4511
18.8522	4511	4750
18.8689	4750	4977
18.8855	4977	5194
18.9022	5194	5402
18.9188	5402	5602
18.9354	5602	5795
18.9521	5795	5981
18.9687	5981	6161
18.9854	6161	6336
19.002	6336	6506
19.0186	6506	6672
19.0353	6672	6834
19.0519	6834	6992

19.0686	6992	7146
19.0852	7146	7297
19.1018	7297	7445
19.1434	7445	1040
19.1493	1040	2080
19.1493	2080	2600
19.1601	2600	3016
19.1767	3016	3374
19.1934	3374	3694
19.21	3694	3986
19.2266	3986	4257
19.2433	4257	4511
19.2599	4511	4750
19.2766	4750	4977
19.2932	4977	5194
19.3098	5194	5402
19.3265	5402	5602
19.3431	5602	5795
19.3598	5795	5981
19.3764	5981	6161
19.393	6161	6336
19.4097	6336	6506
19.4263	6506	6672
19.443	6672	6834
19.4596	6834	6992
19.4762	6992	7146
19.4929	7146	7297
19.5095	7297	7445
19.5262	7445	7590
19.5428	7590	7732
19.5594	7732	7871
19.5761	7871	8008
19.5927	8008	8143
19.6094	8143	8275
19.626	8275	8405
19.6426	8405	8533

19.6593	8533	8659
19.6759	8659	8783
19.6926	8783	8906
19.7092	8906	9027
19.7258	9027	9146
19.7425	9146	9264
19.7591	9264	9380
19.7758	9380	9495
19.7924	9495	9608
19.809	9608	9720
19.8257	9720	9831
19.8423	9831	9941
19.859	9941	10049
19.8756	10049	10156
19.8922	10156	10262
19.9089	10262	10367
19.9255	10367	10471
19.9422	10471	10574
19.9588	10574	10676
19.9754	10676	10777
19.9921	10777	10877

cubic.cwnd

1.00419	0	10400
1.30537	10400	1040
1.31121	1040	7280
1.83785	7280	1040
1.84369	1040	4160
1.84369	4160	5200
1.90441	5200	1040
1.91025	1040	3120
1.91025	3120	4160
4.08426	4160	2080

4.10765	2080	3120
4.22569	3120	1040
4.23153	1040	2080
4.23153	2080	3120
4.58345	3120	1040
4.58929	1040	2080
4.58929	2080	3120
5.09929	3120	1040
5.10513	1040	2080
5.10513	2080	3120
6.09769	3120	1040
6.10353	1040	2080
6.10353	2080	3120
6.17257	3120	1040
6.17841	1040	2080
6.17841	2080	3120
7.44553	3120	1040
7.45137	1040	2080
7.45137	2080	3120
8.36073	3120	1040
8.36657	1040	2080
8.36657	2080	3120
8.90153	3120	1040
8.90737	1040	2080
8.90737	2080	3120
10.5655	3120	1040
10.5714	1040	2080
10.5714	2080	3120
11.8634	3120	1040
11.8693	1040	2080
11.8693	2080	3120
13.6855	3120	1040
13.6914	1040	2080
13.6914	2080	3120
13.7438	3120	1040
13.7496	1040	2080

13.7496	2080	3120
14.7172	3120	1040
14.723	1040	2080
14.723	2080	3120
15.2414	3120	1040
15.2472	1040	2080
15.2472	2080	3120
15.441	3120	1040
15.4469	1040	2080
15.4469	2080	3120
15.8238	3120	1040
15.8296	1040	2080
15.8296	2080	3120
16.4062	3120	1040
16.412	1040	2080
16.412	2080	3120
17.8122	3120	1040
17.8181	1040	2080
17.8181	2080	3120
18.7358	3120	1040
18.7416	1040	2080
18.7416	2080	3120
19.1434	3120	1040
19.1493	1040	2080
19.1493	2080	3120